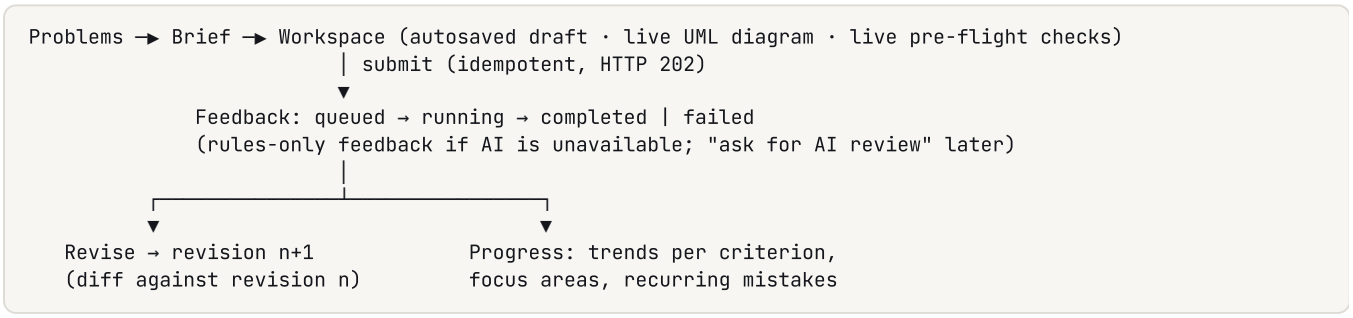
Design note: crit, an LLD practice platform

crit-lld.onrender.com
github.com/Sumit99589/crit-lld

1. The MVP

Four LLD problems (Parking Lot, Vending Machine, Expense Sharing, Elevator), each with numbered requirements, constraints and a **requirement change** the learner must absorb. The learner fills a **design canvas**, and a live UML diagram and live structural checks update as they type. On submit, the design is stored immediately and evaluated in the background. **Deterministic checks** run first, then a **low-cost LLM** (Claude Haiku 5.5) grades six rubric criteria and must quote the design as evidence. The platform verifies every quote, caps grades that contradict hard facts, and shows the evidence underlined in the learner's own text. Revising creates revision 2, whose feedback lists what was fixed, and a progress page shows criteria that keep coming up weak.

2. User flow



3. What a learner provides, and why

Canvas section	Evidence of	Deterministic use
Assumptions	Understanding, handling ambiguity	Present or absent
Classes (name, kind, responsibility, attributes, methods)	Responsibilities, cohesion, abstraction	God-class signals, vague or data-only classes, unimplemented interfaces
Relationships (6 typed kinds)	Coupling, correct relationship kinds	Orphans, ownership cycles, fan-out, “implements” on a class
Requirement map (FR → classes)	Coverage, traceability	Unmapped requirements, one class owning most requirements
Key flow	Behaviour, failure handling	Uses the declared classes? Mentions any failure?
Decisions	Trade-off reasoning	Patterns named without a reason
Response to change	Extensibility	Answered? Grounded in the learner's classes?

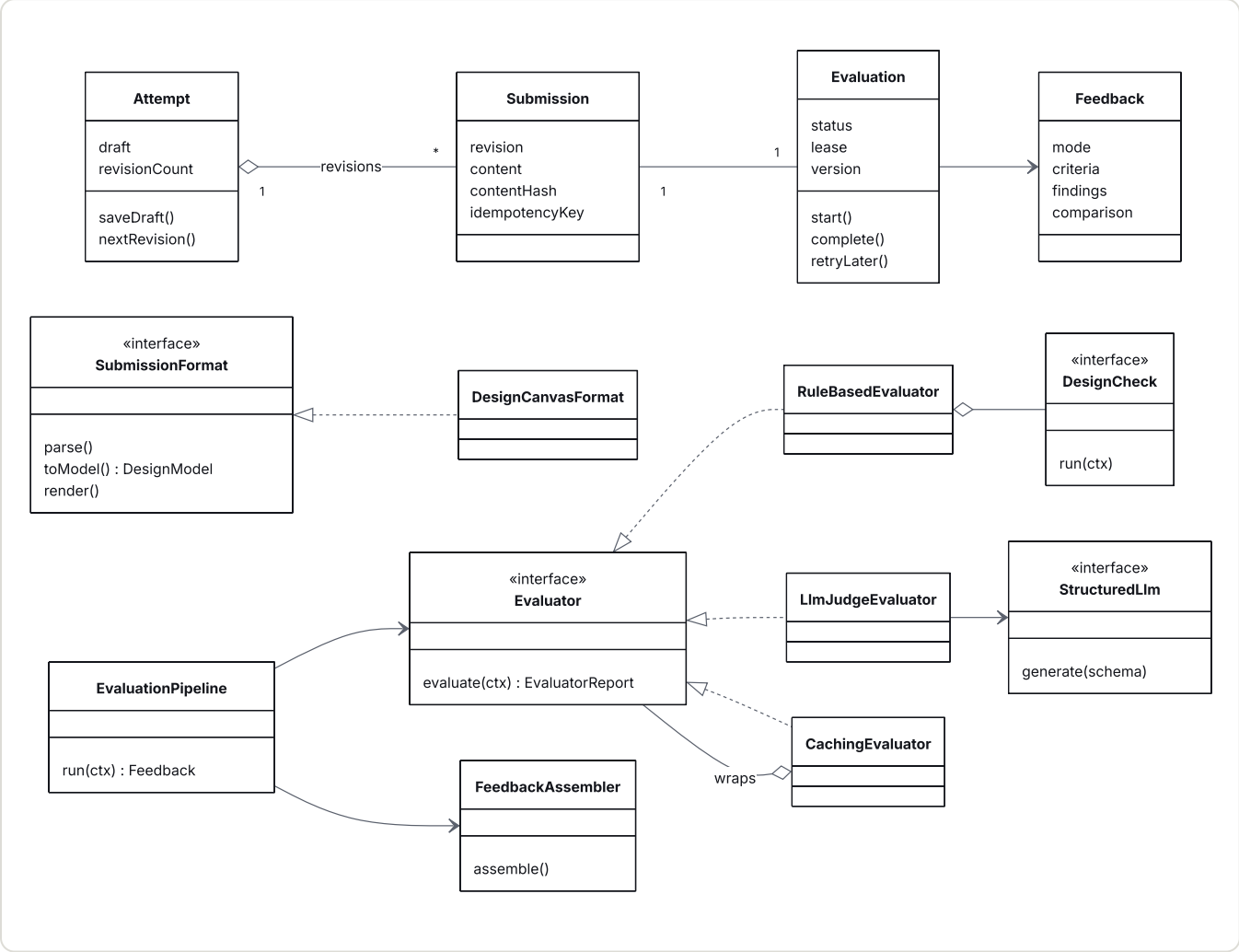
Why a canvas, not code or a diagram? Code proves more but needs a sandbox and per-language parsing. A diagram shows structure but not reasoning. The canvas is the smallest format that captures both *structure* (checkable by rules) and *reasoning* (judgeable by a reviewer), and it renders as a diagram for free. Submission requires only the minimum that makes review meaningful: 3 or more classes with a responsibility sentence, a relationship, and a key flow.

4. Architecture

A single Node/TypeScript monolith with SQLite, serving a React SPA. Dependencies point inward; `container.ts` is the only file that names concrete classes.

```
http/          Express routes, DTO presenters, error → status mapping
application/   PracticeService · EvaluationWorker · ProgressService · ports (repository and queue interfaces)
domain/        Attempt · Submission · Evaluation · Problem · Rubric · Feedback · DesignModel · SubmissionFormat
evaluation/    Evaluator · RuleBasedEvaluator + DesignChecks · LlmJudgeEvaluator · FeedbackAssembler
infra/         SQLite repositories (versioned writes) · InProcessQueue · logger
```

5. Domain model



Class	Owns	Why it exists
Attempt	Mutable draft, revision numbering, ownership	"Try again" is a new revision of the same attempt, so revisions can be compared
Submission	Immutable snapshot plus a content hash	Saved before evaluation, so work is never lost; the hash dedupes identical resubmits
Evaluation	State machine, tries, lease, version	Makes slow or failing evaluation explicit and safe to retry; illegal transitions throw
SubmissionFormat	Parse, readiness rules, → DesignModel, canonical text	**Variation point for new submission formats**
Evaluator	One way of judging a design	**Variation point for new evaluators**
DesignCheck	One deterministic rule	Small and testable; a new rule is one class
FeedbackAssembler	The merge policy	One place decides how rules and AI combine
CachingEvaluator, UsageBudget	Cache by input hash; daily, hourly and lifetime call caps	Keeps cost control out of the judge; a public demo can't run up a bill

Deliberately **not** added: a per-problem rubric (one rubric keeps results comparable), a User aggregate (an anonymous browser id is enough for an MVP), an event bus (a small `JobQueue` port is enough).

6. Evaluation approach

Deterministic (rules)

- Submission readiness (blocking)

Judgement (LLM)

- Quality of each responsibility split

- Requirement coverage from the learner's map
- Ownership cycles, orphans, fan-out, wrong relationship kinds
- God-class signals, vague or data-only classes
- Change scenario answered and grounded
- State transitions, idempotency, dedupe, budget
- Verifying the AI's evidence quotes
- Revision diff and recurring weaknesses
- Abstractions at real points of variation
- Relationship kinds that fit the domain
- Whether the change response actually holds
- Trade-off reasoning, edge cases in the flow
- Strengths and prioritised next steps

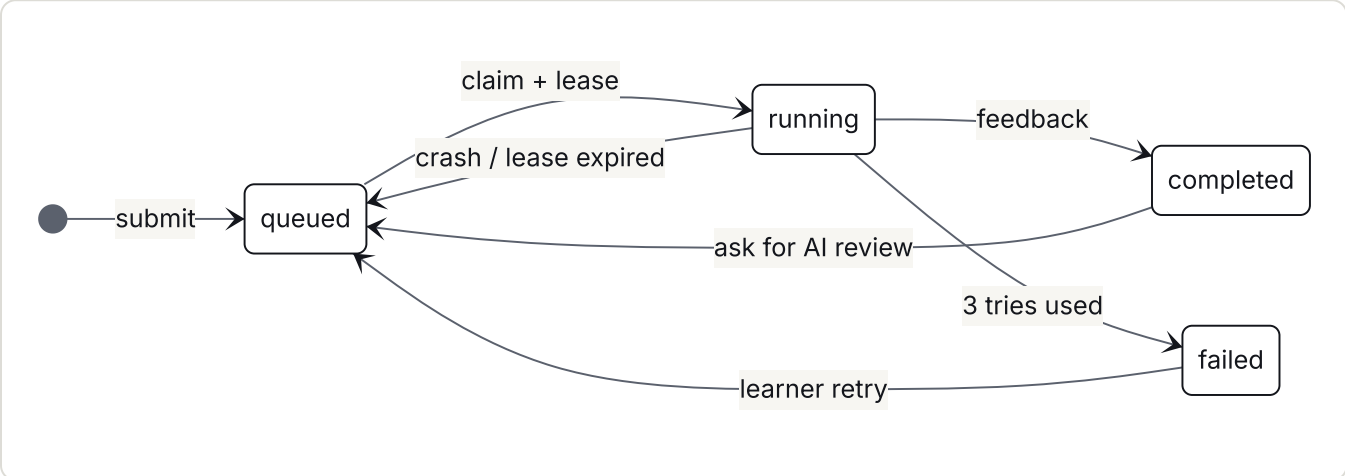
Making AI feedback consistent

- **A fixed rubric:** six criteria × four described levels (Missing / Developing / Solid / Strong). There is no 100-point score.
- **Structured output** per criterion: `evidence → concern → band → suggestion → confidence`. The model quotes and states its concern *before* committing to a band.
- **Rule findings go in as verified facts** the model must not contradict. No reference solution is included; different decompositions are explicitly fine.
- **The submission is untrusted data**, so instructions inside it are ignored. The judge never sees its earlier verdicts (no anchoring), and revision diffs are computed in code.

> **Merge policy.** (1) The AI's assessment wins per criterion, and rules fill any gaps. (2) Every quote is checked against the submission; Solid-or-better grades with no verifiable quote drop to low confidence. (3) Deterministic caps override optimistic bands, with the reason shown. (4) Rule findings are always shown. Rules-only bands never exceed Solid.

Cost: one Haiku call per new design, measured at ~4k input and ~2k output tokens ≈ **$0.0015 per review**. Identical input is cached, and lifetime, daily and per-learner caps degrade to rules-only.

7. When evaluation is slow or fails



- **Never blocks:** submit stores the submission and enqueues it (202), and the UI polls. The submission and its evaluation are written in one transaction and enqueued only after commit.
- **No duplicates:** an `Idempotency-Key` per draft version, plus dedupe on content hash, both backed by unique indexes. Claiming a job is an optimistic `WHERE version = ?` update, so only one worker wins.
- **AI failure** (rate limit, timeout, refusal, bad JSON, no key, budget) completes as `rules_only` with the reason, immediately. Crashes retry with backoff 3 times, then end as `failed` with a retry button. On boot, `recover()` re-queues stale work.

8. The two change tests

A: class-diagram submissions. Add `PlantUmlFormat implements SubmissionFormat` that produces the same `DesignModel`. Attempts, submissions, the worker, every check and the judge are unchanged; checks that need

narrative simply find less. Only the UI needs a new editor.

B: another evaluator or human review. A synchronous evaluator is one more `Evaluator` in the pipeline, and the assembler already merges by criterion. Human review is asynchronous, so it becomes a second `Evaluation` row per submission with `kind = "human"`. The practice flow is untouched; the only schema change is evaluation 1:1 → 1:N by kind.

– 9. Scale, trade-offs and limits

- **Split out the evaluation worker first.** It only needs the DB and a `JobQueue` port, and the lease plus versioned claim make several workers safe. Then move to Postgres and a real queue behind the same ports.
- **Limits:** anonymous browser identity, shallow English-centric heuristics (findings and caps only, never top grades), a judge not yet calibrated against a hand-graded set, polling instead of push, and a single node.
- **Tests:** 62 Vitest/Supertest tests cover the state machine, every check, quote verification, caps, revision diffs, worker failure modes (rate limit → rules-only → retry, crash → failed, lease recovery, concurrent claim, cache, budgets) and the HTTP API, with a fake LLM for determinism.